[GPU 똑똑하게 쓰기 (2)] 알아서 배치하고, 장애에도 이어간다 — Multi-Clustering과 GPU Failover 기술 2026. 10. 14. 16:00

https://blog.naver.com/post6155/224440379799



> GPU가 늘어날수록 Cluster는 GPU 기종과 용도에 따라 나뉘고, 장애는 일상이 됩니다. 여러 클러스터를 하나의 자원 Pool처럼 운영하는 Multi-Clustering과 GPU 장애를 감지해 작업을 살려내는 이상 탐지 및 Failover 기술을 살펴봅니다.

핵심 인사이트

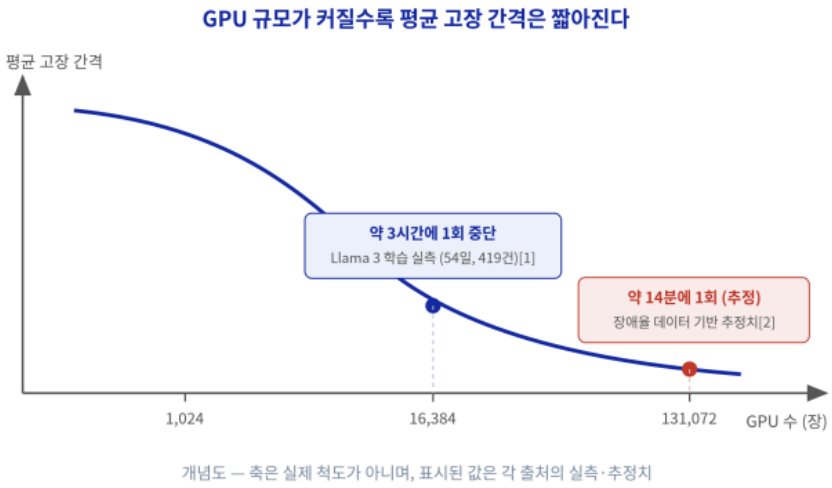
- **다양한 GPU를 하나의 통합 자원처럼:** 멀티클러스터링으로 여러 클러스터의 GPU를 필요에 따라 함께 선택하고 활용합니다.
- **장애, 없앨 수 없다면 손실을 줄여봅시다:** 장애 감지·복구 시간을 줄여 유효 학습 시간 비율(ETTR)을 높여야 합니다.
- **GPU 장애 복구, 사람 손 없이 하기:** 상황에 따른 작업 재시작과 정상 노드·다른 클러스터로의 이전을 자동화하는 Fail-Over 체계가 필요합니다.

GPU가 늘어날수록 커지는 두 가지 과제

GPU를 사용하고 계신다면, 수개월마다 출시되는 신형 GPU를 도입해야 하고 사업이 커질수록 GPU 수가 늘어납니다. 그러다 보면 두 가지 운영 과제를 마주하게 됩니다.

첫 번째 과제, 여러 개의 GPU 클러스터를 잘 운영해야 합니다. 신형 GPU를 도입하거나 여러 CSP(Cloud Service Provider)를 사용한다면 자연히 클러스터의 수가 증가합니다. 학습, 서빙에 따라 최적화된 구성이 다르다면 클러스터를 분리하여 구성하는 것이 유리합니다. 그러나 이렇게 여러 개의 클러스터를 사용하면 어떤 문제가 생길까요? 바로 GPU 활용률이 감소합니다. A 클러스터에 GPU 여유분이 있어도, B 클러스터에 사용자들이 몰려 GPU 사용 대기열이 길어진다면 GPU에 투자한 금액을 제대로 활용하지 못하게 됩니다.

두 번째 과제, 빈번한 장애를 해결해야 합니다. Meta가 공개한 자료에 따르면, H100 16,384장으로 구성된 클러스터에서 Llama 3를 54일간 학습하는 동안 예기치 않은 중단이 419건 발생했습니다. 평균 3시간에 한 번꼴이며, 원인의 절반 이상은 GPU와 GPU 메모리(HBM3)였습니다.[1] 스탠퍼드 연구진은 이런 장애율 데이터를 바탕으로, 131,072장 규모의 작업에서는 평균고장간격(MTBF, Mean Time Between Failures)이 14분까지 짧아질 것으로 추정했습니다.[2] 대규모 GPU 클러스터에서 장애는 확률의 문제가 아니라 주기의 문제입니다.



[그림1] GPU 규모 증가에 따른 평균 고장 간격의 감소
(출처: Meta[1] 및 Gandhi & Kozyrakis[2] 자료 재구성)

장애 한 건의 비용은 적지 않습니다. 학습에서는 마지막 저장 지점 이후의 작업이 되돌려지면서 수백에서 수천 이상의 GPU 시간이 사라질 수 있고, 복구가 지연되면 다른 Job의 대기까지 길어집니다. 서빙에서는 GPU 한 장의 장애가 곧 그 GPU가 들고 있던 세션과 KV 캐시의 유실로 이어집니다.

이번 글에서는 학습까지 포함한 인프라 전체 관점에서 주제를 살펴보고, 이에 대한 판단 기준을 먼저 제시합니다. 앞의 문제(자원의 분산)에 대해서는 GPU 활용률, 뒤의 문제(장애의 일상화)에 대해서는 ETTR(Effective Training Time Ratio)입니다. ETTR은 할당된 총시간 중 실제 학습에 사용된 시간의 비율로, 예를 들어 ETTR 0.9는 10시간 중 9시간이 실제 학습에 쓰이고 1시간은 장애 복구와 오버헤드로 소모되었다는 뜻입니다. 이어지는 내용은 나뉜 클러스터를 하나처럼 쓰는 기술(멀티클러스터 구성과 스케줄링), 그리고 장애가 나도 작업이 이어지게 하는 기술(이상 탐지, Failover, Self Healing)의 순서로 살펴봅니다.

GPU는 남는데, 왜 못 쓸까?

• 클러스터는 왜 늘어나는가

GPU 클러스터가 여러 개로 나뉘는 데에는 몇 가지 이유가 있습니다. 크게 세 가지 정도로 볼 수 있습니다. **먼저 GPU 기종입니다.** GPU는 기종마다 메모리 용량, 연산 성능, 노드 간 네트워크 사양이 다릅니다. 서로 다른 기종의 GPU를 하나의 분산 학습에 섞으면 동기식 학습의 특성상 가장 느린 장비에 학습 처리 속도가 수렴합니다. 즉, 빠른 GPU가 느린 GPU를 기다리게 됩니다. 반대로 동일 기종으로 클러스터를 구성하면 GPU 간 성능 차이가 작아 이런 대기 시간을 줄일 수 있고, 자원을 보다 효율적으로 활용할 수 있습니다. 또한 드라이버, CUDA, 펌웨어, NCCL(NVIDIA Collective Communications Library), 네트워크 설정 등을 동일하게 가져갈 수 있어 문제가 발생했을 때 원인을 파악하고 대응하는 게 수월해집니다.

다음은 용도입니다. 서빙은 사용자의 요청에 빠르게 응답해야 하기 때문에 응답 지연과 트래픽 변화에 대응하기 위해 탄력적인 운영이 필수입니다. 반면, 학습은 수일에서 수 주에 걸쳐 대량의 자원을 안정적으로 사용해야 합니다. 이처럼 운영 방식이 다른 워크로드를 하나의 클러스터에 함께 배치하면 자원 경쟁이 발생하거나 운영 복잡도가 높아질 수 있어, 용도에 따라 클러스터를 분리하는 경우가 많습니다.

마지막은 장애 격리입니다. GPU를 활용하는 서비스가 하나의 클러스터만 사용하게 되면 네트워크 스위치나 스토리지 등 공통 인프라에 문제가 생겼을 때, 서비스에 영향이 생길 수 있습니다. 반대로 클러스터를 여러 개로 나누면 장애가 발생하더라도 영향 범위를 특정 클러스터 안으로 제한할 수 있어 전체 서비스가 동시에 영향을 받는 위험을 줄일 수 있습니다.

이처럼 GPU의 특성과 사용 목적, 장애의 영향 범위까지 고려하면 모든 GPU를 하나의 클러스터로 운영하기보다는 각각의 특성에 맞게 클러스터를 분리해 운영하는 것이 보다 효율적이고 안정적입니다. 그러나 문제는 클러스터를 나눈 이후부터 시작됩니다. 각 클러스터가 독립적으로 운영되면 한쪽에는 작업이 몰리는 반면 다른 쪽에는 자원이 남는 상황이 발생할 수 있습니다. 또한 사용자가 특정 클러스터를 직접 이용하는 구조라면 클러스터를 증설하거나 교체할 때마다 사용자와 운영 시스템에도 영향을 줄 수 있습니다.

글로벌 고객을 상대하는 Uber 역시 일부 클러스터에는 작업이 몰려 스케줄링 지연이 발생하는 동안 다른 클러스터의 자원은 충분히 활용되지 않는 문제를 겪었습니다. 그렇다면 Uber는 이 문제를 어떻게 해결했을까요?

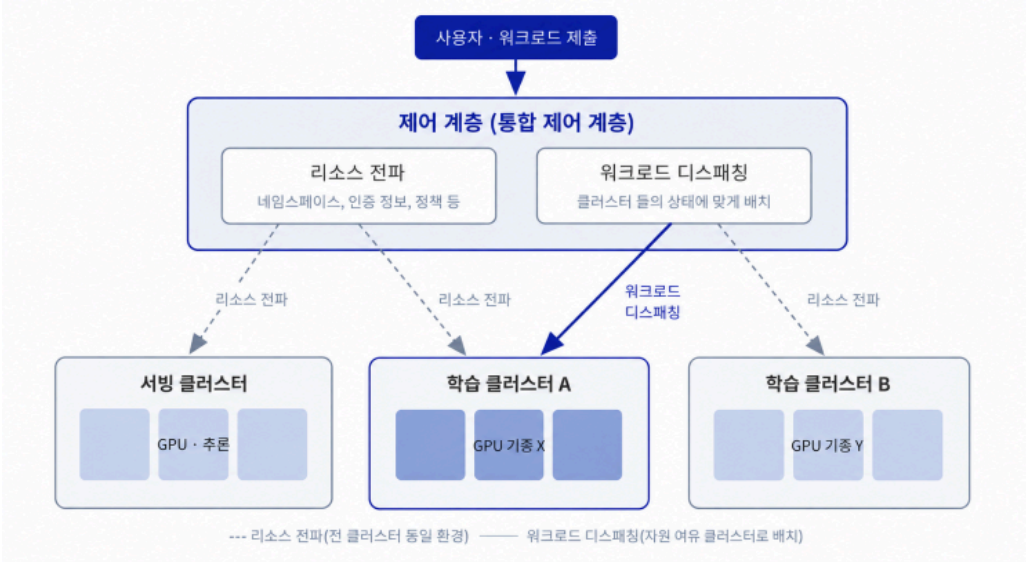
- **해법: 여러 클러스터를 하나처럼 운영하는 통합 제어 계층을 도입한다**

Uber의 해법은 **멀티클러스터링**이었습니다. Federator라는 통합 제어 계층을 도입하여 여러 클러스터를 마치 하나처럼 운영되도록 구성했습니다. 특정 클러스터에 직접 워크로드를 제출하는 대신 Federator에 요청을 보내면, Federator가 여러 클러스터의 자원 가용량과 현재 수요를 확인해 적절한 클러스터를 선택합니다. 이렇게 Uber는 멀티클러스터링으로 여러 클러스터의 부하와 이용률 편차를 줄이고 자원을 보다 효율적으로 활용할 수 있게 되었습니다.

멀티클러스터링의 핵심은 Uber의 Federator처럼 여러 클러스터를 통합적으로 제어하는 제어 계층과 GPU 등 실제 워크로드가 사용할 자원을 보유한 실행 클러스터의 분리입니다. 사용자와 서비스가 제어 계층에 요청을 보내면, 제어 계층이 워커 클러스터들의 자원 상태를 파악해 적절한 곳에 워크로드를 배치합니다. 이 구조가 성립하려면 크게 두 가지 기능이 필요합니다.

첫 번째는 **리소스 전파**입니다. 워크로드가 어느 워커 클러스터로 가더라도 실행될 수 있으려면, 네임스페이스, 인증 정보, 정책 같은 실행 환경이 모든 클러스터에 일관되게 준비되어 있어야 합니다. 이를 위해 제어 계층은 워크로드에 필요한 리소스를 여러 워커 클러스터에 자동으로 전파하고 동기화하는 기능이 필요합니다.

두 번째는 **워크로드 디스패칭**입니다. 사용자가 제어 계층에 워크로드를 제출하면, 제어 계층은 여러 실행 클러스터 중 실행 가능한 대상에 워크로드를 배포합니다. GPU 기종에 따라 실행 클러스터를 구분해 두고 적절한 후보 클러스터로 워크로드를 배치함으로써 여러 GPU 클러스터를 하나의 환경처럼 운영할 수 있습니다.



[그림2] 제어 계층의 리소스 전파와 워크로드 디스패칭 구조 (출처: 삼성SDS)

Samsung Cloud Platform(SCP)의 GPU 서비스는 서빙 클러스터와 기종별 학습 클러스터를 제어 계층인 매니저 클러스터가 통합 관리하는 구조로 운영되고 있으며, 학습 워크로드를 생성하면 리소스를 포함한 사용자 실행 환경이 전 클러스터에 자동으로 전파됩니다. 제어 계층 - 실행 클러스터 구조로 클러스터 간 배치 문제가 정리되면, 다음 문제는 각 클러스터 안에서 워크로드를 어떤 순서로 실행하느냐입니다. GPU 학습 워크로드는 크기 편차가 매우 크기 때문에, 이 순서 결정이 전체 활용률을 좌우합니다.

대규모 학습은 왜 계속 밀릴까?

- **기성 스케줄링 정책이 통하지 않는 이유**

스케줄링은 오래된 문제라 검증된 답도 많습니다. 먼저 온 순서대로 실행하고, 급한 작업에 우선순위를 주고, 자리가 부족하면 덜 급한 작업을 밀어내면 됩니다. 이를 선점(Preemption)이라고 합니다. 밀려난 작업은 언젠가 순서가 돌아오면 다시 시작합니다. 웹서비스 클러스터에서는 이 상식이 잘 동작합니다. 하지만 학습용 GPU 클러스터에 이 정책들을 그대로 적용하면 효율이 급격히 감소합니다. 실행 중인 학습을 밀어내기가 쉽지 않기 때문입니다.

웹서비스는 다시 시작하는 데 필요한 비용이 학습 워크로드만큼 크지 않습니다. 수일째 실행된 학습 워크로드를 중단시키면 마지막 체크포인트 이후의 진행 분이 사라지고, 대량의 GPU가 맞물린 워크로드를 통째로 재기동해야 합니다. 대규모 학습일수록 체크포인트의 사이즈가 커져, 저장하고 복원하는 비용이 함께 늘어납니다. 따라서 선점을 전제로 한 자동 체크포인트/재기동 인프라를 갖추지 못한 환경이라면, 한번 시작한 학습이 완주를 보장받을 수 있도록 무선점 운영이 현실적인 선택이 됩니다.

선점이 없는 환경에서 기성 정책들은 각각 다른 방식으로 실패합니다. 도착 순서대로 실행하는 FIFO에서는 GPU 수백 장을 요구하는 대형 학습이 대기열의 맨 앞에서 자리가 나기를 기다리는 동안, 그 뒤의 소형 학습들도 함께 대기합니다. 남는 GPU가 있어도 쓰지 못하는 것입니다. 반대로 실행 가능한 학습부터 채워 넣는 방식은 소형 학습이 끊임없이 끼어들면서 대형 학습이 영원히 시작되지 못하는 기아(Starvation)를 만듭니다. 예상 실행 시간을 받아 빈틈에 소형 학습을 끼워 넣는 백필(Backfill)은 이론적인 대안이지만, 사용자들이 학습이 중단되는 것을 피하려고 예상 시간을 최대치로 입력하는 경향이 있어 실효성이 떨어집니다.

이 문제들은 클러스터가 여러 개로 나뉘는 순간 더 커집니다. 대형 학습은 결국 한 곳에서만 실행될 텐데, 모든 대기열의 맨 앞에 동시에 줄을 서면 곳곳의 빈 GPU까지 일제히 잠깁니다. 밀어낼 수도 없고, 순서대로 세워도, 빈틈을 채워도 실패한다면 어떻게 해야 할까요?

- **해법: 배치는 알아서, 자원은 규모별 큐로 나눠 융통한다**

스케줄러의 역할을 배치와 배분 두 단계로 나누면 됩니다. 배치는 학습을 어느 클러스터로 보낼지 판단하는 단계입니다. 사용자는 워크로드를 제출할 뿐, 실행할 클러스터를 선택하지는 않습니다. 스케줄러가 클러스터를 구성하는 노드들의 남은 GPU 숫자를 파악하여, 규모에 맞는 자리를 골라 보냅니다. 이때 소형 학습은 이미 학습으로 붐비는 클러스터에 모아 채우고, 빈 노드가 많은 곳은 대형 학습의 몫으로 남겨 둡니다. 작은 학습이 큰 학습의 자리를 조각내지 않도록 배치 단계에서 미리 정리해 두는 것입니다.

배분은 학습이 특정 클러스터에 배치된 이후, 같은 클러스터에 모인 학습을 어떤 순서로 실행할지 정하는 단계입니다. 규모에 따라 큐를 대형, 소형으로 나누고, 두 큐 사이의 자원 배분을 동적으로 조정합니다. 대형 큐에 대기가 쌓이면 소형 큐가 빌려 쓰던 자원을 거둬들여 진입로를 확보하고, 대형 큐가 한가해지면 소형 큐가 유휴 자원을 쓰도록 풀어 줍니다. 큐 안에서는 대기 시간과 요구 GPU 수를 반영해 우선순위를 주기적으로 끌어올려 대형 학습의 기아 상태로 진입하지 않게 보정합니다.

요점은 하나의 완벽한 규칙을 찾는 것이 아니라, 대형 학습의 진입 보장과 소형 학습의 빠른 처리라는 상충하는 목표 사이에서 배분 비율을 운영 데이터에 따라 계속 조정할 수 있는 구조를 갖추는 데 있습니다. 크고 작은 학습을 일렬로 세워 두는 대신 각자의 궤도에 올려 간섭 없이 돌게 하는 셈입니다.



[그림3] Job 규모별 큐 이원화와 대기 상황에 따른 동적 자원 배분 (출처: 삼성SDS)

배치 판단의 재료는 자원 상태에서 데이터 위치로 확장되고 있습니다. 학습에는 데이터 세트와 체크포인트가 따라다니는데, 이 데이터가 없는 클러스터에 학습을 배치하면 실행에 앞서 대용량 전송이 먼저 발생합니다. 이 또한 비용 증가의 원인입니다. 클러스터별 스토리지에 필요한 데이터가 있는지를 배치 판단에 반영하는 Data-aware 스케줄링이 필요한 이유이며, 이는 뒤에서 다룰 클러스터 간 Failover의 전제 조건이기도 합니다. 멀티클러스터 구성과 스케줄링으로 분산된 GPU를 하나의 자원 Pool처럼 쓸 수 있게 되어도, 그 Pool을 구성하는 GPU 한 장 한 장이 고장 나는 문제는 남습니다. 그리고 앞서 설명한 것처럼 규모가 커질수록 이 문제는 더 자주 발생합니다.

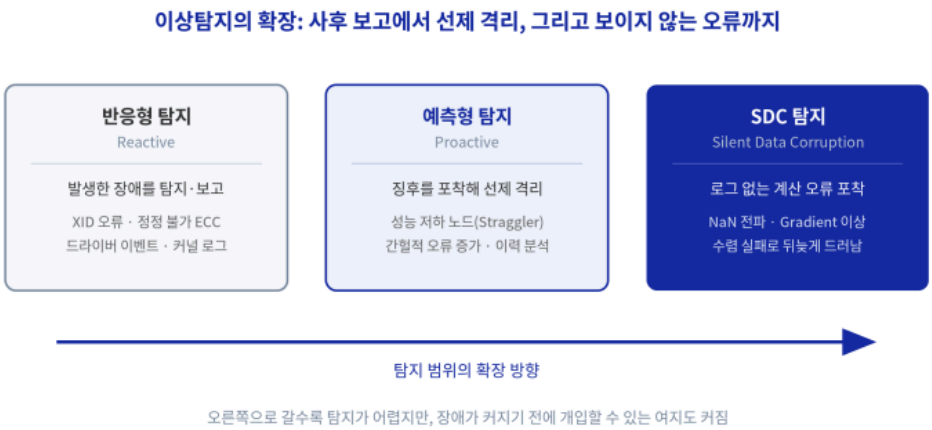
GPU는 정상으로 보이는데, 학습은 왜 멈추고 결과는 이상해질까?

- **장애는 감지가 절반이다**

GPU 장애가 까다로운 것은 발생 빈도 때문만이 아니라 드러나는 방식 때문입니다. 세 가지 유형으로 나눌 수 있습니다. 명시적 장애는 그나마 다루기 쉽습니다. GPU 메모리의 정정 불가능한 오류(ECC Error, Error Correction Code), GPU가 시스템에서 인식되지 않는 상태, 과열이나 전원 이상처럼 오류 신호가 명확히 남는 경우입니다. NVIDIA GPU 드라이버는 이런 오류에 XID라는 코드를 부여해 기록하며, DCGM(NVIDIA Data Center GPU Manager) 같은 진단 도구로 수집할 수 있습니다.[4]

연쇄 장애는 원인 찾기가 어렵습니다. 분산학습에서는 수백 개의 프로세스가 집합 통신으로 동기화되는데, NIC(Network Interface Card) 한 개가 데이터를 전송하지 못하면 그와 통신하던 프로세스들이 먼저 멈추고, 이들을 기다리던 프로세스들이 연쇄적으로 멈추면서 수 초 안에 전체가 정지 상태에 빠집니다. 겉으로는 모두가 멈춰 있어서, 최초 원인이 어느 노드인지 로그만으로는 가려내기 어렵습니다.

가장 까다로운 것은 조용한 데이터 손상(SDC, Silent Data Corruption)입니다. 오류 로그도 없고 프로세스도 정상 동작하지만, 하드웨어 결함으로 계산 결과 자체가 틀리는 경우입니다. 그 영향이 그래디언트(Gradient)와 모델 파라미터에 누적되어 Loss 급등이나 수렴 저하로 뒤늦게 나타날 수 있으며, 일부는 비정상 수치(NaN이나 Inf)로 드러나기도 합니다. 이런 특성 때문에 업계의 이상 탐지는 발생한 장애를 보고하는 반응형(Reactive) 탐지에서, 성능 저하나 간헐적 오류 증가 같은 징후를 포착해 장애 전에 노드를 격리하는 예측형(Proactive) 탐지로 확장되고 있습니다. Meta는 주기적인 헬스 체크와 과거 장애 데이터를 결합해 성능이 저하된 노드(Lemon Node)를 선제 격리하고, 정지 상태가 감지되면 프로세스 간 대기 관계를 역추적해 최초 원인 노드를 찾아내는 분석 체계를 운영한다고 밝혔습니다.[3]



[그림4] 이상 탐지의 확장: 반응형 탐지에서 예측형 탐지, SDC 탐지까지 (출처: 삼성SDS)

- **판단 기준: 옮겨야 할 장애와 옮겨도 소용없는 오류**

탐지만큼 중요한 것이 분류입니다. 같은 "학습 중단"이라도 원인이 하드웨어인지 소프트웨어인지에 따라 대응이 정반대이기 때문입니다. GPU 메모리의 정정 불가능한 오류나 GPU 인식 불가처럼 그 노드에서 계속할 수 없는 장애라면 다른 노드로 옮겨야 합니다. 반면 잘못된 메모리 접근처럼 애플리케이션에서 비롯된 오류는 노드를 옮겨도 같은 문제가 반복되므로, 재배치가 아니라 사용자에게 원인을 알려주는 것이 올바른 대응입니다. 같은 OOM(Out Of Memory)이라도 일시적인 메모리 사용량 증가로 발생했다면 재시작으로 해소될 수 있지만, 배치 크기 설정처럼 작업 자체가 원인이면 몇 번을 다시 시작해도 같은 지점에서 멈춥니다.

분류	대표 사례	대응
하드웨어 장애	정정 불가 ECC 오류(XID 48), GPU 인식 불가(XID 79), SRAM 오류(XID 94/95), 과열·전원 이상	노드 격리 후 정상 자원으로 재배치
소프트웨어·일시적 오류	NCCL 타임아웃, 일시적 메모리 스파이크로 인한 OOM	같은 노드에서 재시작
사용자·설정 오류	코드 버그로 인한 메모리 페이지 폴트(XID 31), 연산 예외(XID 13), 설정 문제	재배치 거부, 원인 로그 제공

*XID는 NVIDIA GPU 드라이버의 오류 코드

[표1] 학습 중단 원인의 분류와 대응 방향 (출처: 삼성SDS)

Samsung Cloud Platform의 GPU 클러스터는 진단 도구와 커널 모니터링을 상시 구동하여 GPU 하드웨어부터 네트워크, 소프트웨어 설정까지 다양한 이상 유형을 탐지합니다. 몇 시간씩 걸리던 장애 원인 분석도 분 단위로 끌어 내리고 있습니다. 이상을 감지하고 분류했다면, 남은 일은 고장 난 GPU에서 실행 중이던 작업을 살려내는 것입니다.

GPU 한 장이 고장 나면, 수천 장의 학습을 처음부터 다시 해야 할까?

- **재시작에서 클러스터 간 이전까지, 복구의 3단계**

장애 대응을 "노드를 바꿔 다시 시작한다"라는 단일 동작으로 이해하면 복구 비용이 불필요하게 커집니다. 실제 복구는 오류의 성격과 심각도에 따라 단계적으로 설계됩니다. **가장 가벼운 단계는 제자리 재시작(In-Place Restart)입니다.** 일시적 메모리 스파이크로 인한 OOM이나 통신 타임아웃처럼 하드웨어가 정상인 오류라면, 노드를 옮기지 않고 같은 자리에서 컨테이너만 다시 시작하는 것이 가장 빠릅니다.

하드웨어 장애로 판정되면 클러스터 내 Failover로 넘어갑니다. 장애 노드를 스케줄링 대상에서 제외(Drain)해 격리하고, Job을 정상 노드로 재배치해 학습을 재개합니다. 이때 재배치되는 Job에 높은 우선순위를 부여해 대기열에서 오래 기다리지 않게 합니다. 다만 재배치 시 GPU 메모리에 있던 학습 상태는 유지되지 않으므로, 공유 스토리지에 저장된 체크포인트에서 이어서 시작하게 됩니다. 지난 편에서 다룬 Concurrent Checkpointing이 바로 이 저장 오버헤드를 줄이는 기술입니다.

복구 속도를 끌어올리는 보조 장치들도 함께 발전하고 있습니다. 사전 검증을 마친 예비 노드 Pool을 준비해 두었다가 장애 즉시 투입하는 Warm Standby, 코드나 설정 변경 시 실행 환경을 파괴하지 않고 적용해 재시작 비용을 줄이는 In-Place Hot-Update가 대표적입니다. TikTok으로 유명한 ByteDance는 이런 기법들을 결합한 관리 시스템으로 9,600장 GPU에서 3개월간 진행한 학습에서 ETTR 97%를 달성했다고 발표했습니다.[5]

마지막 단계가 클러스터 간(Cross-Cluster) Failover입니다. 클러스터 내 가용 GPU가 소진되었거나, 네트워크 스위치 장애로 다수의 노드가 한꺼번에 이탈해 클러스터 안에서는 복구할 수 없는 상황이라면, Job을 다른 클러스터로 옮겨 재개해야 합니다. 앞서 살펴본 멀티클러스터 구조가 여기서 복원력의 기반이 됩니다. 매

니저 클러스터는 전체 워커 클러스터의 자원 상태를 이미 알고 있으므로 이전할 곳을 고를 수 있고, 리소스 전파로 실행 환경이 준비되어 있으므로 Job을 받을 수 있습니다. 남는 조건은 데이터입니다. 체크포인트와 데이터 세트가 대상 클러스터에서 접근 가능해야 하므로, 클러스터 간 스토리지 복제 또는 공유 스토리지 구성이 클러스터 간 Failover의 실질적인 전제 조건이 됩니다.

하이퍼스케일러들의 복원력 기능과 비교하면 이 마지막 단계의 필요성을 더 명확히 알 수 있습니다. AWS SageMaker HyperPod는 상시 헬스 체크와 자동 노드 교체, 마지막 체크포인트에서의 자동 재개(Auto-resume)를 제공하고[6], NVIDIA는 NeMo 프레임워크의 Resiliency 확장으로 장애 시 자동 재개와 지연 노드 탐지를 지원합니다.[7] 이들은 수천에서 수만 장 규모의 단일 클러스터를 전제로 하는 클러스터 내 복구에 초점을 두고 있습니다. GPU가 기종별로 여러 클러스터에 분산된 환경이라면, 클러스터 경계를 넘는 작업 이전까지 복구 범위에 포함하는 설계가 필요합니다.

- **Self Healing: 복구의 마지막 조각**

작업을 옮기는 것으로 끝나면 절반의 복구입니다. 격리된 장애 노드가 방치되면 클러스터의 가용 용량이 줄어든 채 유지되고, 결국 어느 시점에는 옮길 곳이 없어집니다. 그래서 Failover의 마지막 조각은 장애 노드를 진단하고 조치해 다시 클러스터로 되돌리는 절차입니다. 운영 관점의 흐름은 이렇습니다. 이상 탐지가 노드를 장애 상태로 표시하면 운영자는 탐지 로그에서 원인을 확인하고, 부품 교체나 드라이버 복구 같은 조치가 필요하면 노드를 클러스터에서 제외한 상태로 작업합니다. 조치가 끝난 노드는 검증을 거쳐 클러스터에 다시 합류하고, 스케줄링 대상으로 복귀합니다. 감지, 격리, 작업 이전, 노드 복구, 복귀로 이어지는 이 순환이 자동화될수록 사람의 개입과 GPU 불용 시간이 줄어듭니다. 사용자 입장에서 장애가 있었는지조차 모르게 만드는 것이 이 순환의 목표입니다.



[그림5] 장애 감지부터 노드 복귀까지의 Self Healing 순환 (출처: 삼성SDS)

이 기술들을 실제 GPU 운영에 어떻게 적용할까?

지금까지 살펴본 기술이 실제 서비스에 적용된 사례 중 하나가 Samsung Cloud Platform의 Simple AI Training 서비스입니다.[8] 멀티클러스터 구성과 스케줄링은 학습 서비스의 기반 구조로 동작하고 있습니다. 서빙 클러스터와 기종별 학습 클러스터를 매니저 클러스터가 통합 관리하며, 사용자는 클러스터의 존재를 의식할 필요 없이 Job을 제출하면 자원 상태에 따라 적합한 클러스터에 배치됩니다. Job 규모에 따른 큐 운영으로 대형 학습과 소형 실험이 한 자원 Pool에서 공존할 수 있도록 지원합니다.

이상 탐지와 Failover는 Simple AI Training의 Job Failover 기능으로 제공되고 있습니다. 사용자가 Training Job 생성 시 Failover를 활성화하면, 시스템이 학습 노드의 GPU와 하드웨어 상태를 지속적으로 점검하다가 하드웨어 장애로 판정된 경우 장애 노드를 피해 정상 자원으로 Job을 자동 재배치합니다. 앞서 다룬 분류 기준이 그대로 적용되어, 사용자 코드 오류나 설정 문제로 판단되면 재배치를 수행하지 않고 원인을 로그로 남깁니다. 같은 문제가 반복될 재배치로 사용자의 시간과 자원이 낭비되는 것을 막기 위한 설계입니다. 장애로 중단된 학습과 재개된 학습의 로그는 하나의 스트림에서 이어서 확인할 수 있어, 장애 전후의 흐름을 놓치지 않고 추적할 수 있습니다.



[그림6] GPU 클러스터 스케줄링과 GPU Job Failover를 제공하는 Simple AI Training 서비스 사례
(출처: 삼성SDS)

이 구조에서 사용자가 얻는 것은 단순합니다. 장애 감시, 원인 분류, 노드 교체, 재스케줄링이라는 운영 부담을 플랫폼에 맡기고, 체크포인트 저장 전략 같은 학습 코드 수준의 준비에만 집중하면 됩니다. 지난 리포트에서 소개한 유휴 GPU의 Spot 활용과 이번 편의 Failover가 결합되면, 자원을 놀리지 않으면서도 장애에 끊기지 않는 운영에 한 걸음 더 다가설 수 있습니다.

마치며

멀티클러스터 구성과 스케줄링은 기종과 용도에 따라 나뉜 GPU를 하나의 자원 Pool로 만들어 활용률을 끌어올리고, 이상 탐지와 Failover, Self Healing은 그 Pool이 장애에도 끊기지 않도록 유효가동시간(ETTR)을 지켜냅니다. 확보한 GPU에서 얼마나 많은 작업을 얼마나 끊김 없이 처리하느냐가 AI 인프라의 원가와 품질을 함께 결정하기 때문에, 두 축은 함께 갖춰질 때 의미가 있습니다.

지금까지 SCP GPUaaS 연재를 통해 워크로드 정합성에서 시작해 인퍼런스 서비스, 라우팅과 캐시, 학습으로의 확장을 거쳐 복원력까지, GPU를 사용한 만큼만 내는 서비스가 성립하기 위한 기술 조건들을 살펴봤습니다. 만약 자체 GPU 인프라를 운영하고 있다면 다음 순서의 점검을 권합니다. 클러스터 분리 기준과 스케줄링 정책이 Job 규모 편차를 감당하는지 확인하고, 학습 코드의 체크포인트 저장 주기와 저장 위치를 점검한 뒤, 이상 탐지가 반응형을 넘어 예측형까지 커버하는지, 그리고 장애 판정과 재배치 기준, 재시도 한계가 정의되

어 있는지 확인하는 것입니다. 이 네 가지가 갖춰져 있다면, 장애가 잦아지는 규모에서도 GPU는 계속 일하게 만들 수 있습니다.

References

[1] Meta AI. (2024). The Llama 3 Herd of Models. arXiv:2407.21783 (16,384 H100, 54일, 중단 419건)

[2] Gandhi, S., & Kozyrakis, C. (2024). MoEtion: Efficient and Reliable Sparse Checkpointing for Mixture-of-Experts Models at Scale. arXiv:2412.15411

[3] Kokolis, A., et al. (2024). Revisiting Reliability in Large-Scale Machine Learning Research Clusters. arXiv:2410.21680

[4] NVIDIA. XID Errors, GPU Deployment and Management Documentation. https://docs.nvidia.com/deploy/xid-errors/

[5] Wan, B., et al. (2025). Robust LLM Training Infrastructure at ByteDance. SOSP '25. arXiv:2509.16293

[6] AWS. Amazon SageMaker HyperPod Resiliency. https://docs.aws.amazon.com/sagemaker/latest/dg/sagemaker-hyperpod-resiliency.html

[7] NVIDIA. NeMo Framework Resiliency. https://docs.nvidia.com/nemo-framework/user-guide/latest/resiliency.html

[8] Samsung SDS. Simple AI Training, Samsung Cloud Platform 서비스 포탈. https://cloud.samsungsds.com/serviceportal/services/ai_ml/simpleAItraining.html

▶ 이 아티클은 AI를 활용하여 제작되었습니다.

- 톤앤 매너 보정 및 도식화 : Claude

FAQ

Q1. 멀티클러스터 구성은 언제부터 필요합니까?

GPU 기종이 두 세대 이상 섞이거나, 학습과 서빙을 같은 자원에서 운영하게 되는 시점이 일반적인 분기점입니다. 세대가 다른 GPU를 하나의 분산학습에 섞으면 가장 느린 장비에 전체 속도가 묶이고, 서빙과 학습은 운영 요건이 상반되기 때문입니다. 클러스터를 나눈 뒤에는 이용률 불균형을 막기 위한 통합 관리 계층이 필요해집니다.

Q2. ETTR이란 무엇입니까?

Effective Training Time Ratio의 약자로, 작업에 할당된 총시간 중 실제 학습에 사용된 시간의 비율입니다. 장애 복구, 재시작, 체크포인트 복원 등에 쓰인 시간이 많을수록 낮아집니다. 대규모 학습 인프라의 복원력을 평가하는 실질적인 지표로 쓰이고 있습니다.

Q3. Failover와 단순 재시작은 무엇이 다릅니까?

단순 재시작은 같은 노드에서 작업을 다시 실행하는 것으로, 일시적 소프트웨어 오류에 적합합니다.
Failover는 하드웨어 장애로 판정된 노드를 격리하고 작업을 정상 노드나 다른 클러스터로 옮겨 재개하는 것입니다. 원인 분류가 선행되어야 두 대응을 올바르게 나눌 수 있습니다.

Q4. 어떤 오류가 Failover 대상이 됩니까?

정정 불가능한 GPU 메모리 오류, GPU 인식 불가, 과열·전원 이상처럼 해당 노드에서 작업을 계속할 수 없는 하드웨어 장애가 대상입니다. 반면 학습 코드의 메모리 초과나 버그로 인한 오류는 노드를 옮겨도 반복되므로 재배치 대상에서 제외되고, 원인 로그가 사용자에게 제공됩니다.

Q5. 체크포인트 없이 Failover가 수행되면 어떻게 됩니까?

재배치 시 GPU 메모리의 학습 상태는 유지되지 않으므로, 저장된 체크포인트가 없으면 작업이 처음부터 다시 시작될 수 있습니다. Failover 기능과 별개로, 공유 스토리지에 주기적으로 체크포인트를 저장하도록 학습 스크립트를 구성하는 것을 권장합니다.